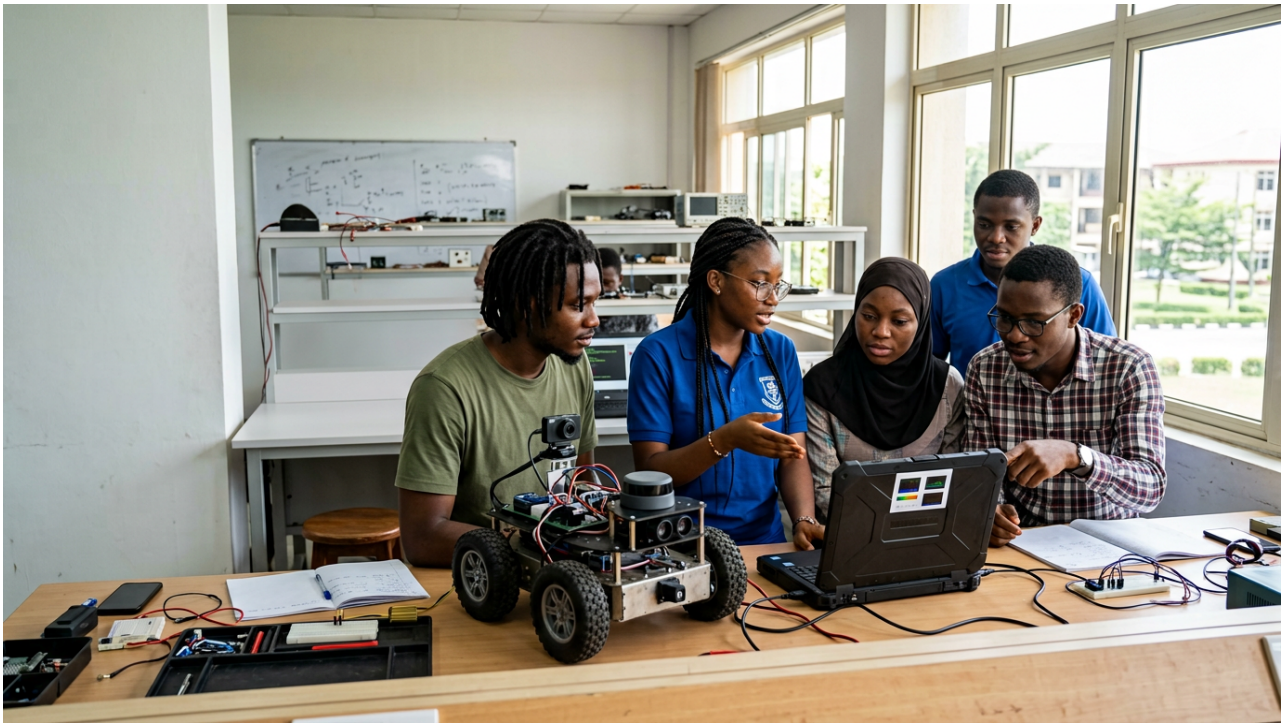
AI.101 — Introduction to Artificial Intelligence



Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.

Narration manuscript · Module 10: Natural Language and Conversational AI

Academic level: Bachelor of Artificial Intelligence and Robotics Engineering
Content policy: No faith-related content.

1. Language is structured, contextual and ambiguous

Natural language processing, or NLP, concerns computational methods for analyzing, representing and generating human language. A text string is not merely a sequence of isolated words. Meaning depends on context, word order, reference, domain conventions and what a speaker is trying to accomplish. The same phrase can be a request, a quotation, a joke, a warning or an instruction embedded in a document.

Consider the sentence, “The robot stopped near the door because it was blocked.” What does “it” refer to—the robot or the door? A human reader uses context and world knowledge to infer a likely meaning, but a program must process available evidence. In safety-relevant settings, ambiguity should sometimes trigger a clarification question rather than a confident guess.

Language systems are used for search, translation, document classification, speech interfaces, summarization and conversational assistance. These tasks differ. A search engine retrieves documents. A classifier assigns categories. A summarizer produces a shorter representation. A conversational model generates responses across turns. A single product may combine these tasks, but each component needs its own evaluation.

2. A basic text-processing pipeline

A simple NLP pipeline may begin with text collection and cleaning. The system might normalize whitespace, handle punctuation, detect language and remove or preserve special tokens. These choices depend on the task. Removing punctuation can help some bag-of-words models but destroy useful information for sentence boundaries or code-like text.

Tokenization divides text into units. A word-based tokenizer splits at spaces and punctuation. Subword tokenizers divide uncommon words into pieces. Modern language models often work with token IDs rather than complete words. A word may use several tokens; a token may represent a word part, punctuation or a frequent sequence. Tokenization affects context length, language coverage and model behavior.

A text system then converts tokens into a representation it can compute with. A simple bag-of-words representation counts terms and ignores much of their order. An embedding maps tokens or sequences to vectors whose geometry reflects learned relationships. A contextual representation changes depending on surrounding tokens. These representations can support classification or generation, but they do not provide a complete explanation of meaning.

Raw sentence → normalized text → tokens → numeric representation → task-specific model → output. Annotate which decisions are reversible and where information may be lost.

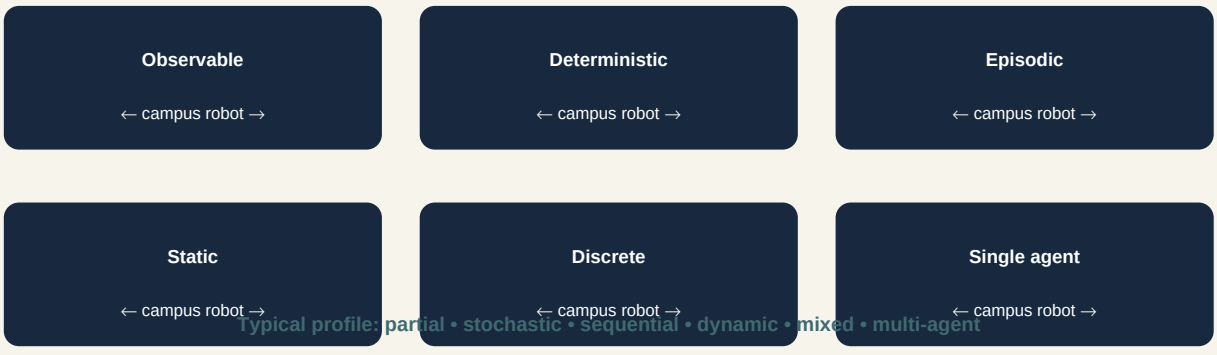


Diagram: Environment classification profile

3. Classification and retrieval

Text classification assigns labels to a text, such as positive or negative sentiment, support request category, or document topic. A simple baseline might use keywords or term frequencies with logistic regression. More advanced systems use contextual neural representations. Evaluation should include per-class precision and recall, especially if classes are imbalanced or errors have different consequences.

Information retrieval ranks documents according to relevance to a query. A search system can use lexical overlap, semantic embeddings, metadata filters, or combinations. Retrieval returns candidate evidence; it does not automatically guarantee that a generated answer accurately reflects the sources. The system should preserve the link between a claim and the passages that support it.

A useful assistant architecture can retrieve from a bounded, approved document collection before drafting an answer. It can show citations and allow the user to inspect the source. If no relevant evidence is found, it should say so. The user interface should distinguish directly quoted text from generated paraphrase and clearly indicate when an answer is uncertain.

4. Language models and generated text

A language model estimates patterns in token sequences. A generative model produces output by selecting likely next tokens given the preceding context and its learned parameters. Repeating this process creates a sequence. Sampling settings influence how predictable or varied the output is, but no setting turns a language model into a guaranteed fact-checker.

A response can be grammatically fluent and factually unsupported. The model may combine familiar phrases in a way that sounds coherent while making an incorrect claim. It may omit an important condition, misunderstand the user’s goal, or produce a citation that does not exist. The model has learned statistical patterns; unless linked to a verified source and tested, it cannot be assumed to know whether a statement is true in the current situation.

Conversational systems also need dialogue state. They must track relevant parts of earlier turns, resolve references, identify when the user changes topic, and manage context limits. A system may forget or compress earlier details. It should not imply that it remembers information it no longer has available. In high-impact applications, designers should limit the assistant’s scope and make uncertainty visible.

5. Prompt injection and untrusted content

A conversational system may receive instructions from users, documents, retrieved web pages, or tool outputs. These sources should not all have equal authority. A retrieved document may contain text that looks like an instruction to the assistant. The system must treat external content as data to analyze, not automatically as a trusted command.

Prompt injection is one example of a broader security problem: an attacker attempts to manipulate model behavior through crafted input. Defenses include separating instruction and data channels, limiting tool privileges, validating tool arguments, restricting sensitive operations, using allowlists, and monitoring outputs. No single prompt phrase guarantees security. If a model can trigger physical or financial actions, an independent policy and approval layer should constrain what actions are permitted.

6. Worked design: a technical-manual assistant

Imagine a technician asking, “What should I check if the motor temperature rises during startup?” A poor assistant may immediately produce a plausible checklist from its learned patterns. A better bounded system first searches the approved manual and maintenance records, filters for the correct motor model and revision, and identifies relevant passages. It drafts a response tied to those passages, distinguishes required checks from optional suggestions, and asks for missing information such as the model number if needed.

The system should be evaluated with questions whose answers are present, absent, ambiguous or contradictory in the document collection. It should be tested against outdated manuals, mismatched models, irrelevant search results and malicious instructions hidden in a document. Reviewers should verify that the cited passages actually support each claim.

The assistant must also respect safety procedures. If the requested task involves electrical isolation or hazardous motion, it should refer to the approved procedure and require human confirmation rather than improvising. The system should not bypass a safety interlock because a user phrases a request forcefully.

7. Measuring language-system quality

No single metric fully measures a conversational response. Relevance asks whether it addresses the question. Factuality asks whether claims are supported. Completeness asks whether important conditions are included. Clarity asks whether the intended user can understand it. Safety asks whether the response could cause harm or expose sensitive information. Citation quality asks whether evidence exists and supports the claim.

Automated text-similarity scores can be useful for narrow comparisons, but many valid responses use different wording. Human evaluation can examine meaning and consequences, but reviewers need clear criteria and sufficient subject expertise. For a technical assistant, include domain experts in the evaluation. Record disagreement and uncertainty rather than forcing reviewers to pretend every case has one obvious label.

Test diverse phrasing, spelling variation, short prompts, long context, multiple languages where supported, refusal cases and out-of-scope questions. Evaluate consistency across repeated runs. If the model is updated, repeat relevant tests; changing the model or retrieval index can change behavior.

8. Practical laboratory: retrieval-grounded question answering

Create a small assistant over a short set of non-sensitive technical documents. Split documents into chunks with metadata such as title, revision and section. Implement lexical retrieval as a baseline; optionally compare it with vector retrieval. For each query, display the selected passages before the generated answer.

Require the response to cite its evidence and to state when the supplied materials do not contain the answer. Build a test set with answerable, unanswerable, ambiguous and conflicting questions. For each generated answer, check that cited passages exist, the answer follows from those passages, and no unsupported details were added. Log failures and revise the retrieval or response process.

Do not connect this prototype to equipment controls. Keep the system read-only. If you add a tool call, use a mock function that cannot alter a real system. In your report, describe data handling, document permissions, prompt-injection risks, evaluation methods and human oversight.

9. Common mistakes

A frequent mistake is to equate fluent language with understanding or factual reliability. Another is to use overall text similarity as the only quality measure. A third is to treat retrieved text as trusted instructions. A fourth is to include sensitive documents without considering access control and retention. A fifth is to allow a conversational model to perform consequential actions without permission checks or a human review path.

10. Knowledge check and assignment

How does text classification differ from information retrieval? Why does tokenization matter? Why can a fluent response still be false? How can retrieval and citations improve a technical assistant, and what problems do they not solve? What is prompt injection, and why are tool permissions important?

For the assignment, build and evaluate a read-only document assistant. Submit a system diagram, data inventory, retrieval baseline, test questions, evidence checks, failure analysis and a short safety case. Include at least one example where the correct system behavior is to ask a clarifying question or say that the evidence is insufficient.

Module summary

NLP systems process and generate language through pipelines of normalization, tokenization, representation, retrieval and inference. We separated classification, retrieval and generation; explained the basics of language models; and examined ambiguity, unsupported outputs, prompt injection and evaluation. A responsible conversational system is bounded, evidence-aware, tested against failure cases, careful with data and restricted from unreviewed consequential actions.